

# Compiling the finetuned GutVLM to ONNX

Cross-platform (CPU) inference for the Mobile-O Gut-VLM understanding model — engineering report

Prepared by Claude Code • 23 July 2026 • repo: Gut\_Vlm\_finetune / Mobile-O-GutVLM-App

## 1. Executive summary

The finetuned GutVLM model (Mobile-O, a LLaVA-style Qwen2 vision-language model trained for gastrointestinal-endoscopy VQA and hallucination detection) previously ran only on Apple Silicon Macs, via a Core ML + MLX export. This work compiles the same finetuned checkpoint to **ONNX** so it runs on the plain CPU of **any** laptop — Windows, Linux, Intel-Mac or Apple-Mac — through ONNX Runtime, with no GPU, no Apple frameworks, and no PyTorch at inference time.

The understanding pass was split into **three ONNX graphs** (vision encoder + projector, token embedder, and the Qwen2 decoder with a KV cache). The vision encoder is kept in FP32 and the language model is quantized to **weight-only INT8**. The resulting model is ~1.1 GB and reproduces the original PyTorch model **token-for-token** on the tested prompts. All artifacts were verified numerically and uploaded to Hugging Face for one-command download.

## 2. Was this grounded in your codebase? — Yes.

Every design decision below was taken **after reading the actual source files** in the repository (and the finetuned checkpoint's own config and weight names), not from generic assumptions about VLMs. The table lists each file that was read and what it contributed to the ONNX build.

| Source file read                                   | What it told me / how it was used                                                                                                                                                                                        |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mobile-O-GutVLM-App/export.py                      | The existing Core ML + MLX export. Established the 2-component pattern (vision, LLM+projector), the mm_projector weight remapping, and the rationale for defaulting the LLM to 8-bit (precision-sensitive medical task). |
| Mobile-O-GutVLM-App/mlx_infer.py                   | The reference runtime: image preprocessing (resize 1024, no ImageNet mean/std), tokenizer_image_token, the <image> splice logic, and greedy decode. onnx_infer.py mirrors it one-to-one.                                 |
| Mobile-O/inference.py                              | The PyTorch ground-truth ask() / detect_hallucinations(), the exact prompt formats and the two-turn hallucination protocol — copied verbatim.                                                                            |
| .../multimodal_llava_encoder/mobileclip_encoder.py | The vision tower (MobileCLIP-L / FastViT @ 1024), feature_select() producing [1,256,3072], and the CLIPImageProcessor parameters.                                                                                        |
| .../language_model/mobileo_inference.py            | The model class (Qwen2ForCausalLM + LlavaMeta), how generate() builds inputs_embeds via prepare_inputs_labels_for_multimodal.                                                                                            |
| .../model/llava_arch.py                            | The accessor API: get_model(), get_vision_tower(), mm_projector, encode_images() — used to build the export wrappers.                                                                                                    |

checkpoint config.json +  
model.safetensors keys

Exact architecture numbers and tensor shapes (below), so the standalone  
decoder matches bit-for-bit.

### 3. The model, as read from the code

GutVLM's understanding path (the generation / diffusion stack is never touched) is three stages. All numbers below were taken from the checkpoint's config.json and verified against the safetensors tensor shapes.

| Stage          | What it is                                            | Input → Output                                    |
|----------------|-------------------------------------------------------|---------------------------------------------------|
| Vision encoder | MobileCLIP-L / FastViT @ 1024×1024                    | pixel_values [1,3,1024,1024] → feats [1,256,3072] |
| MM projector   | mlp2x_gelu: Linear(3072→896) → GELU → Linear(896→896) | [1,256,3072] → [1,256,896]                        |
| LLM            | Qwen2-0.5B decoder + lm_head                          | inputs_embeds [1,L,896] → logits [1,L,151649]     |

**Qwen2 configuration:** hidden\_size 896, 24 layers, 14 attention heads, 2 KV heads (grouped-query attention, head\_dim 64), intermediate\_size 4864, SiLU MLP, RMSNorm eps 1e-6, RoPE theta 1,000,000, vocab 151,649, separate (untied) lm\_head, q/k/v biases present. This is textbook Qwen2 — important, because it meant the decoder could be re-implemented cleanly (see Challenge C).

The runtime trick, taken from mlx\_infer.py: tokenize the prompt around a literal <image> marker, embed the text, run vision+projector, then **splice** [text-before, image-embeds, text-after] into one embedding sequence and decode autoregressively.

### 4. The ONNX compilation approach

Because the image embeddings must be spliced *between* two text spans, the token embedder has to be callable on its own from the host. So the model was compiled to **three** ONNX graphs (the Core ML/MLX build got away with two only because MLX exposed the embedder as a Python module):

| ONNX graph          | Signature                                                                          | Precision          | Size   |
|---------------------|------------------------------------------------------------------------------------|--------------------|--------|
| vision_encoder.onnx | pixel_values → image_embeds [1,256,896] (vision tower + projector folded together) | FP32               | 505 MB |
| embed_tokens.onnx   | input_ids [1,n] → text_embeds [1,n,896]                                            | INT8               | 136 MB |
| decoder.onnx        | inputs_embeds + position_ids + past_kv → logits + present_kv (Qwen2, KV cache)     | INT8 (weight-only) | 510 MB |

**Host-side inference loop** (onnx\_infer.py) reproduces mlx\_infer.py exactly: preprocess → vision → tokenize with <image> sentinel → embed text spans → splice → **prefill** the decoder (empty past) → greedy **decode** step-by-step, feeding each step's present KV cache back in as the next past.

#### Quantization scheme: FP32 vision + weight-only INT8 LLM

This was your explicit choice (the “fp32 + int8 vs int8” decision). The vision encoder stays FP32 because hallucination detection is precision-sensitive — the same argument export.py makes for the MLX build defaulting to 8-bit rather than 4-bit. The LLM is INT8. **Crucially**, it is *weight-only* INT8 (see Challenge D).

## 5. Challenges faced and how they were solved

### Challenge A — transformers 5.x KV-cache does not trace to ONNX

**Problem:** the HuggingFace Qwen2 model uses the new Cache API, which does not export cleanly through torch's ONNX tracer with an explicit past/present KV cache.

**Solution:** a small, self-contained Qwen2 decoder was re-implemented in plain PyTorch (RMSNorm, RoPE, grouped-query attention, SwiGLU MLP) with an explicit, ONNX-friendly flat KV cache — then **numerically verified against the real HuggingFace forward pass** before being trusted:

```
--- Verifying standalone decoder against HF forward ---
max abs logit diff: 3.099e-05 (rel 2.018e-06)
argmax agreement: 100.0%
VERIFY: PASS
```

This check (on by default in `onnx_export.py`) guarantees the ONNX decoder is the same model, just in an exportable form.

### Challenge B — missing onnxscript

**Problem:** torch 2.12's default ONNX exporter needs the `onnxscript` package, which was not installed.

**Solution:** installed `onnxscript` (a one-time environment fix).

### Challenge C — the exporter could not handle the KV-cache signature

**Problem:** the new torch.export-based exporter could not map `dynamic_axes` onto the decoder's variadic `*past_key_values` inputs, and it wrote model weights to separate external `.onnx.data` files (two files per model).

**Solution:** switched to the legacy TorchScript exporter (`dynamo=False`), which handles the 48-input KV-cache signature and the dynamic sequence/past-length axes correctly, and produces a single self-contained `.onnx` file per graph.

### Challenge D — INT8 quantization broke the language model (the big one)

**Problem:** the first INT8 build used ONNX Runtime's `quantize_dynamic`. VQA still worked ("polyp"), but hallucination detection produced degenerate, repetitive text. Diagnosis showed `quantize_dynamic` also quantizes *activations* (per-tensor), and Qwen2's activation outliers blow that up:

| Metric (long hallucination prompt) | quantize_dynamic (bad) | weight-only INT8 (fix) |
|------------------------------------|------------------------|------------------------|
| max  ONNX logit – FP32 logit       | 9.63                   | 0.31                   |
| prefill next-token argmax          | correct ('The')        | correct ('The')        |
| 30-token greedy generation         | diverges → degenerate  | matches FP32 exactly   |

The tell-tale sign: the very first token was correct (so vision, embed, splice, prefill and the cache were all fine) but greedy decoding forked onto a wrong-but-fluent path a few tokens in — classic accumulated activation-quantization error.

**Solution:** quantize the decoder **weight-only** with `MatMulNBitsQuantizer` (block-wise, symmetric, 8-bit) — weights are INT8 but activations stay FP32. Logit error dropped from 9.63 to 0.31 and generation became identical to FP32. The embedder is a pure lookup (Gather), so INT8 there is safe. This fix is baked into `onnx_export.py`.

### Challenge E — Hugging Face upload returned 403

**Problem:** the first upload failed with 403 Forbidden — the cached login token was read-only, and the fine-grained token in the training repo was scoped to a personal namespace, not the GutVLMmodels organization. **Solution:** used an organization-scoped write token, supplied at run time only and never written into any file, to push to GutVLMmodels/experiments\_checkpoints/gutvlm\_epoch4\_onnx/.

## 6. Verification & results

The ONNX pipeline was compared directly against the original PyTorch checkpoint on the same demo endoscopy image:

| Test                 | PyTorch (reference)                                           | ONNX (fp32 vision + int8 LLM) |
|----------------------|---------------------------------------------------------------|-------------------------------|
| VQA: abnormalities?  | polyp                                                         | polyp — identical             |
| Hallucination detect | ...bleeding. <hallucinated> / ...frame.<br><non-hallucinated> | identical, token-for-token    |
| Correction turn      | adds 'no signs of inflammation...'                            | identical                     |

**Result: full parity.** The cross-platform ONNX model behaves the same as the trained model, while running on ordinary CPUs. Total download  $\approx$  1.1 GB (vision 505 MB + embed 136 MB + decoder 510 MB) plus the tokenizer.

## 7. How to run it locally

### Option 1 — the Gradio app (easiest, for end users)

On **any** laptop (Windows / Linux / Mac). The model auto-downloads from Hugging Face on first run — no manual file handling:

```
pip install -r requirements-onnx.txt gradio
python onnx_app.py
# opens http://localhost:7860 — two tabs: VQA and Hallucination Detection
```

To use a local export instead of downloading, pass `--onnx-dir onnx_models --hf-repo ""`. Use `--threads N` to tune CPU threads, `--share` for a public link.

### Option 2 — command line

```
python onnx_infer.py --onnx-dir onnx_models \
  --image endoscopy.jpg \
  --question "Are there any abnormalities in the image?"

python onnx_infer.py --onnx-dir onnx_models \
  --image endoscopy.jpg --mode hallucination \
  --caption "The image shows a large polyp with active bleeding."
```

### Option 3 — from Python

```
from onnx_infer import load_state, ask, detect_hallucinations
state = load_state("onnx_models")          # or a downloaded folder
print(ask(state, "img.jpg", "Is there a polyp visible?"))
detection, correction = detect_hallucinations(state, "img.jpg", caption)
```

### Re-exporting from a checkpoint (needs PyTorch)

Run from a directory where `import mobileo` resolves to a package that can load the checkpoint (e.g. the Mobile-O training repo):

```
pip install torch onnx onnxscript onnxruntime
python Mobile-O-GutVLM-App/onnx_export.py \
    checkpoints/vlm_gutvlm_hal/epoch_4 \
    --output-dir Mobile-O-GutVLM-App/onnx_models
# flags: --no-quantize (FP32 all) | --quantize-vision | --skip-verify
```

## Deliverables

| File                                         | Purpose                                                        |
|----------------------------------------------|----------------------------------------------------------------|
| onnx_export.py                               | Checkpoint → 3 ONNX graphs, with built-in numeric verification |
| onnx_infer.py                                | Cross-platform runtime (mirrors inference.py / mlx_infer.py)   |
| onnx_app.py                                  | Gradio UI; auto-downloads the model from Hugging Face          |
| requirements-onnx.txt                        | Minimal runtime deps (onnxruntime, transformers, pillow)       |
| README-ONNX.md                               | Full written documentation                                     |
| HF: GutVLMmodels/experiments_check<br>points | Folder gutvlm_epoch4_onnx/ — the hosted model                  |

Generated for the Gut\_Vlm\_finetune project. The ONNX build lives entirely under Mobile-O/Mobile-O-GutVLM-App/ alongside the existing MLX build; the 1.1 GB weights are git-ignored and hosted on Hugging Face.